

Quiz 1:

```
int magicfun( int N) {  
    if ( N == 0)  
        return 0;  
    else  
        return magicfun(N/2) * 10 + (N %  
2);  
}
```

$n = 7$

T.C $\approx (\log(N))$ $N/8$

$N/4$

$N/2$

N

magicfun(0)

magicfun(1)

magicfun(3)

magicfun(7)

$$0 * 10 + (1 \% 2) = 1$$

$$1 * 10 + (3 \% 2) = 10 + 1 = 11$$

$$11 * 10 + (7 \% 2) = 110 + 1 = 111$$

Ans = 111

Quiz 2

```
void fun(char s[], int x) {
```

```
    ① print(s)
    ② char temp
    ③ if(x < s.length/2) {
    ④     temp=s[x]
    ⑤     s[x] = s[s.length-x-1]
    ⑥     s[s.length-x-1]=temp
    ⑦     fun(s, x+1)
    }
}
```

fun("scroll", 0)
⇒

$n/2$ — 6 character

① S C R O L L
L C R O L S
② — —
L L R O C S
③ — —
L L O R C S

Output

```
SCROLL
LCROLS
LLROCS
LLORCS
```

```

X "LLORCS", 3
X "LLROCS", 2
X "LCROLS", 1
"SCROLL", 0
```

$n/2$
 $O(n/2)$
 $= O(n)$

$n/2$ (2) of the string.
contact

T.C $O(n)$

return n. $O(n/2)$

S.C = $O(n)$

$n/2$

Subset

[1, 2, 4]

↳ continuous.
↳ Subsequence: (1, 2) (2, 4)
(1, 2, 4)

Subset

↳ []

[1, 2, 4]

1, 2 1, 4

[2, 1] ×

[4, 1] ×

[1]

[1 2]

[1 4]

[2]

[2 4]

[4]

[1, 2, 4]

* order should
= be same.

* Subset.

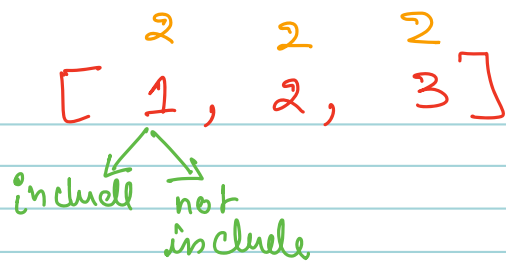
* In subset you can skip an element

* The order should be maintain

* Subsequence / Subarray.

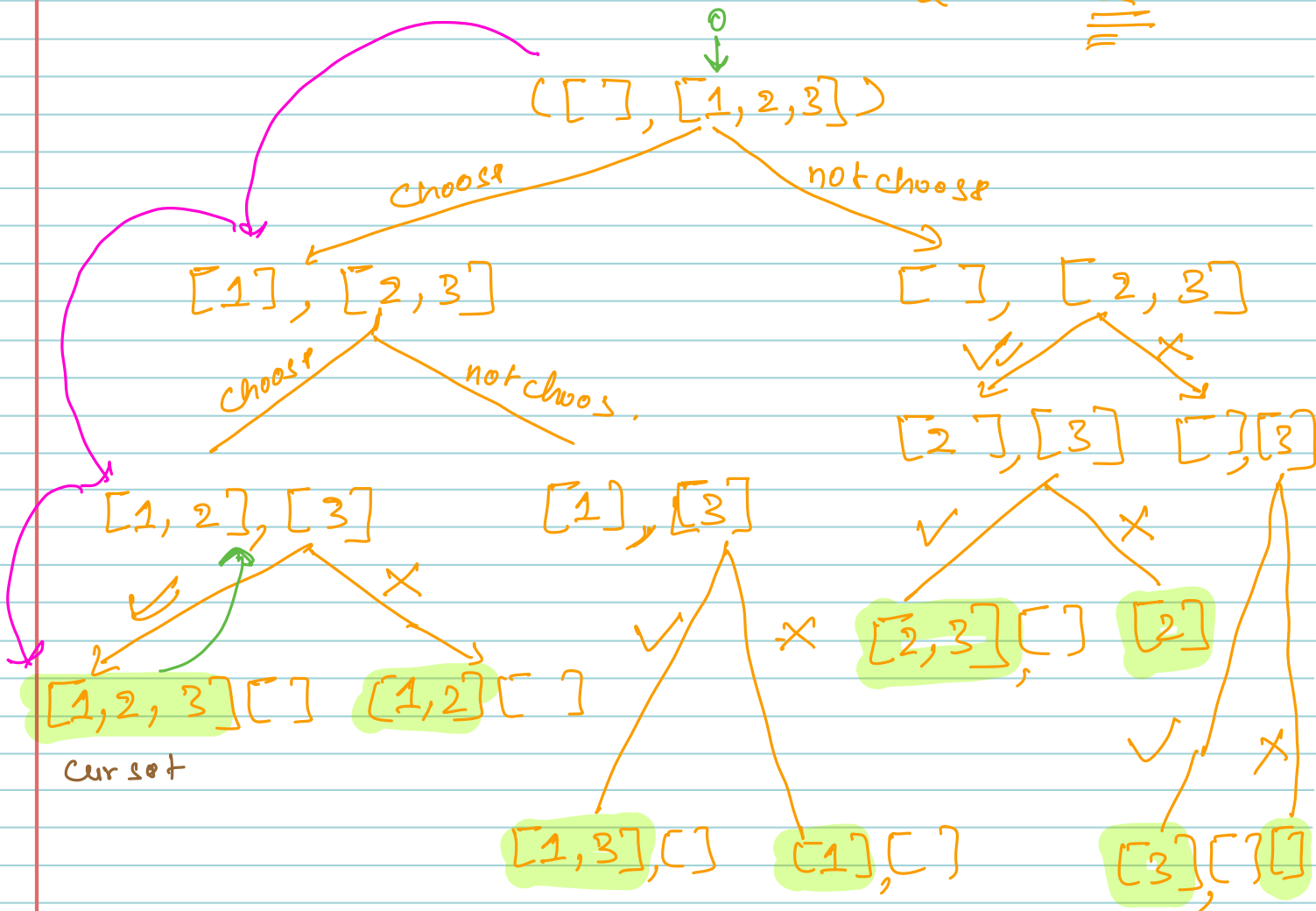
* It should be continuous in the array.

* It is a part of array.



no. of subset = 2^n .

$2^3 = 2^n$



$A = [1, 2, 3],$ idx, curr set
 list int List

[1, 2, 3]

void subsets (list<int> A, int idx, list<int> curset)

1 {

if (idx == A.size())

print (curset); →

return;

// 1: Choose the element

curset.add (A[idx]);

subset (A, idx+1, curset);

6) curset.remove_back();

// 2: Not chose the element.

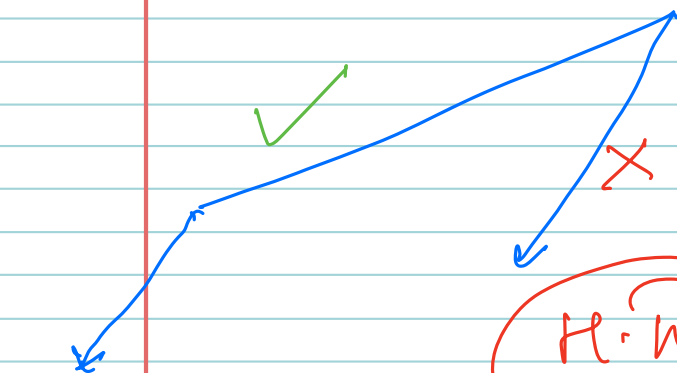
7) subset (A, idx+1, curset);

}

[1, 2, 3] → A

0 → idx

[] → curset.

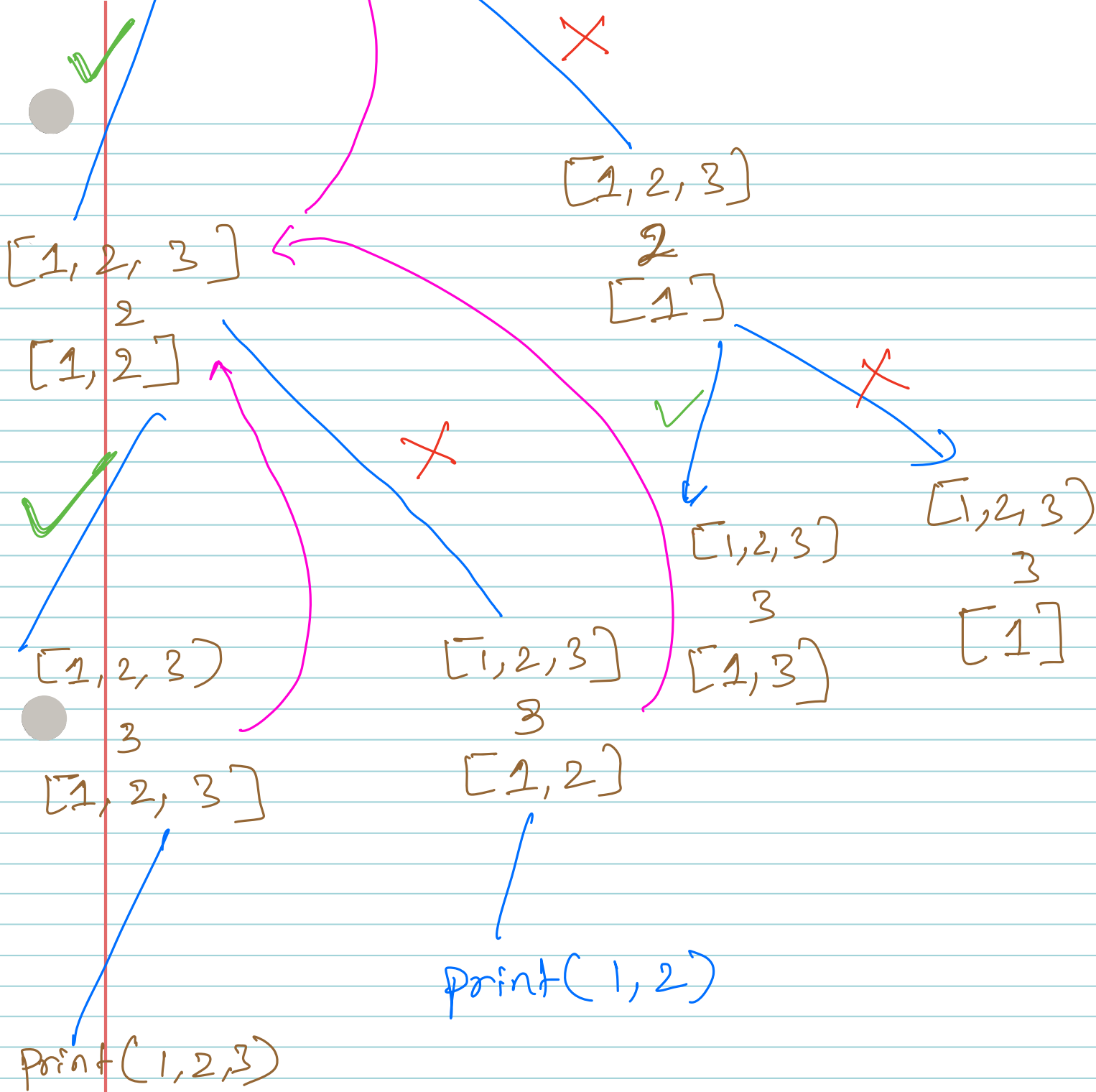


[1, 2, 3]

[1]

H.W.

* primitive
in stack
* array heap



PERMUTATION //

P P Y U S H

$$\rightarrow \underline{6} \times \underline{5} \times \underline{4} \times \underline{3} \times \underline{2} \times \underline{1} = \underline{\underline{6!}}$$

↑

\$f\$ we have \$n\$ unique character
Total Permutation = $n!$

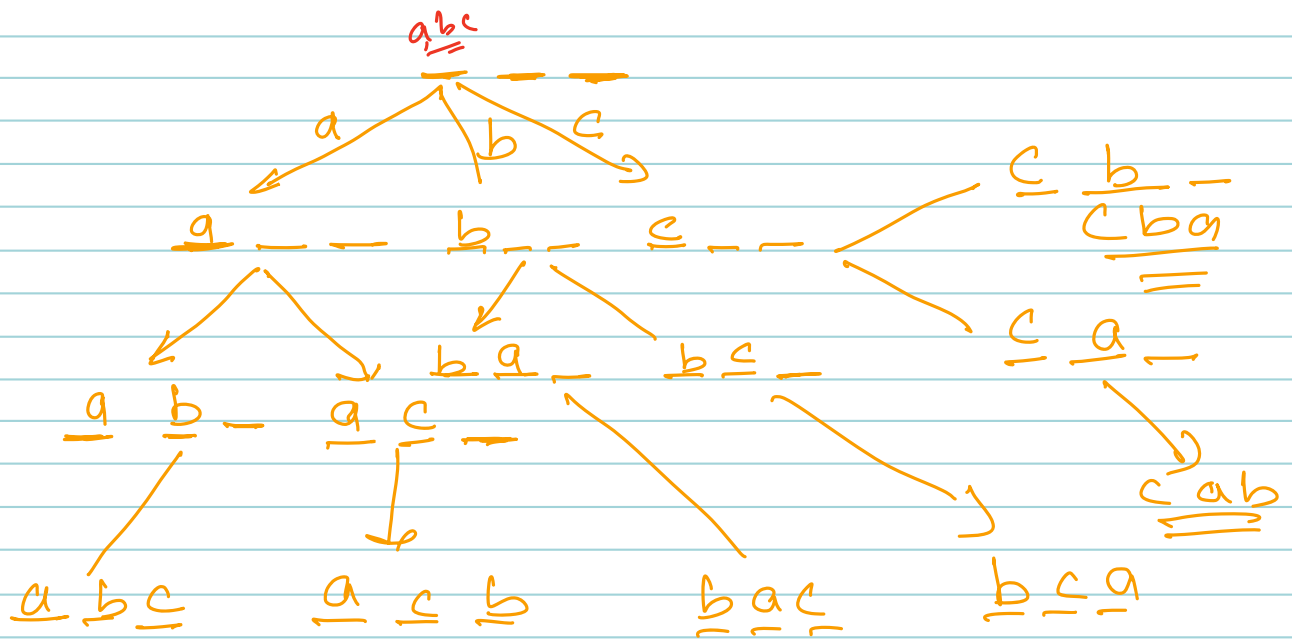
Que Given a string with distinct element. Find all the permutations of the string,

output ✓

a	b	c
a	c	b
b	a	c
b	c	a
c	a	b
c	b	a

} = 3! = 6

a b c



a b c

F F F

F → i haven't look at this c

T → i have looked at character

a - -

[T F F]

next

a b -

[T T F]

a - c -

[T F T]


```
void permutation ( arr, idx, ans[N],  
visited[N])
```

```
{ if (idx == arr.length)
```

```
{ print (ans);
```

```
return;
```

```
} for (i=0; i < N; i++)
```

```
{
```

```
if (visited[i] == false)
```

```
{ visited[i] = true;
```

```
ans[idx] = A[i];
```

```
permutation ( A, idx+1, ans, visited);
```

```
visited[i] = false;
```

```
}
```

```
}
```

```
}
```

→ saving state.

→ bounding → base caseⁿ

50q under

✓

→
understanding & no. of ques

Ques

Valid parentheses.

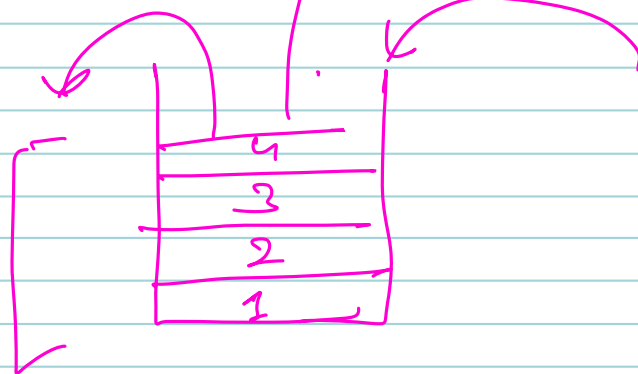
$s =$ "((()))"

only this type.

recursion

Stack

last

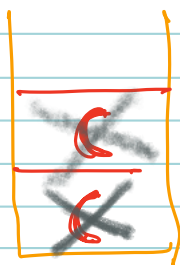


" () () "



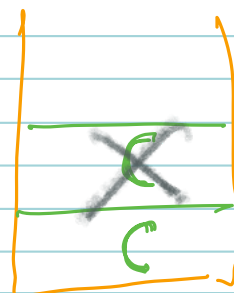
if opening put
in stack

if closing pop
stack.



if stack is empty,
then we it is valid.

" (() "



if stack not
empty it is
not valid.

